LUCRARE DE LABORATOR NR. 1

Tema: Analiza algoritmilor (Timpul de execuţie al algoritmilor).

Scopul lucrării:

1. Analiza empirică a algoritmilor.
2. Analiza teoretică a algoritmilor.
3. Determinarea complexităţii temporale şi asimptotice a algoritmilor

Note de curs:

1. Timpul de execuţie al algoritmilor.

De multe ori, pentru rezolvarea unei probleme, trebuie ales un algoritm dintre mai mulţi posibili, două criterii principale de alegere fiind contradictorii:

1) algoritmul să fie simplu de înţeles, de codificat şi de depanat;

2) algoritmul să folosească eficient resursele calculatorului, să aibă un timp de execuţie redus.

Dacă programul care se scrie trebuie rulat de un număr mic de ori, prima cerinţă este mai importantă; în această situaţie, timpul de punere la punct a programului e mai important decât timpul lui de rulare, deci trebuie aleasă varianta cea mai simplă a programului.

Daca programul urmează a fi rulat de un număr mare de ori, având şi un număr mare de date de prelucrat, trebuie ales algoritmul care duce la o execuţie mai rapidă. Chiar în această situaţie, ar trebui implementat mai înainte algoritmul mai simplu şi calculată reducerea de timp de execuţie pe care ar aduce-o implementarea algoritmului complex.

Timpul de rulare al unui program depinde de următorii factori:

- datele de intrare;
- calitatea codului generat de compilator;
- natura şi viteza de execuţie a instrucţiunilor programului;
- complexitatea algoritmului care stă la baza programului.

Deci timpul de rulare e o funcţie de intrarea sa, de cele mai multe ori, nedepinzând de valorile de la intrare, ci de numărul de date.

In continuare vom nota cu $T(n)$ timpul de execuţie al unui algoritm destinat rezolvării unei probleme de dimensiune n. Pentru a estima timpul de execuţie trebuie stabilit un *model de calcul* şi o unitate de măsură. Vom consideră un model de calcul (numit şi maşină de calcul cu acces aleator) caracterizat prin:

- Prelucrările se efectuează în mod secvenţial.
- Operaţiile *elementare* sunt efectuate în timp constant *indiferent* de valoarea operanzilor.
- Timpul de acces la informaţie nu depinde de poziţia acesteia (nu sunt diferenţe între prelucrarea primului element şi cea a ultimului element al unui tablou).

A stabili o unitate de măsură înseamnă a stabili care sunt operaţiile elementare şi a considera ca unitate de măsură timpul de execuţie a acestora. In acest fel timpul de execuţie va fi exprimat prin numărul de operaţii elementare executate. Operaţiile elementare sunt cele aritmetice (adunare, scădere, înmulţire, împărţire), comparaţiile şi cele logice (negaţie, conjuncţie şi disjuncţie). Cum scopul calculului timpului de execuţie este de a permite compararea algoritmilor, uneori este suficient să se contorizeze doar anumite tipuri de operaţii elementare, numite *operaţii de bază* (de exemplu în cazul unui algoritm de căutare sau de sortare se pot contoriza doar operaţiile de comparare) şi/sau să se considere că timpul de execuţie a acestora este unitar (deşi operaţiile de înmulţire şi împărţire sunt mai costisitoare decât cele de adunare şi scădere în analiza se poate considera că ele au acelaşi cost).

Timpul de execuţie al întregului algoritm se obţine însumând timpii de execuţie a prelucrărilor componente.

Importanta celui mai defavorabil caz. In aprecierea şi compararea algoritmilor interesează în special cel mai defavorabil caz deoarece furnizează cel mai mare timp de execuţie relativ la *orice* date de intrare de dimensiune fixă. Pe de altă parte pentru anumiţi algoritmi cazul cel mai defavorabil este relativ frecvent.

În ceea ce priveşte analiza celui mai favorabil caz, aceasta furnizează o margine inferioară a timpului de execuţie şi poate fi utilă pentru a identifica algoritmi ineficienţi (dacă un algoritm are un cost mare în cel mai favorabil caz, atunci el nu poate fi considerat o soluţie acceptabilă).

Timp mediu de execuţie. Uneori, cazurile extreme (cel mai defavorabil şi cel mai favorabil) se întâlnesc rar, astfel ca analiza acestor cazuri nu furnizează suficientă informaţie despre algoritm.

In aceste situaţii este utilă o altă măsură a complexităţii algoritmilor şi anume *timpul mediu de execuţie.* Acesta reprezintă o valoare medie a timpilor de execuţie calculată în raport cu distribuţia de probabilitate corespunzătoare spaţiului datelor de intrare.

2. Ordin de creştere

Pentru a aprecia eficienţa unui algoritm nu este necesară cunoaşterea expresiei detaliate a timpului de execuţie. Mai degrabă interesează modul în care timpul de execuţie creşte o dată cu creşterea dimensiunii problemei. O măsură utilă în acest sens este *ordinul de creştere.* Acesta este determinat de *termenul dominant* din expresia timpului de execuţie. Când dimensiunea problemei este mare valoarea termenului dominant depăşeşte semnificativ valorile celorlalţi termeni astfel că aceştia din urmă pot fi neglijaţi.

Întrucât problema eficienţei devine critică pentru probleme de dimensiuni mari se face analiza complexităţii pentru cazul când n este mare (teoretic se consideră că $n \longrightarrow \infty$), în felul acesta luându-se în considerare doar comportarea termenului dominant. Acest tip de analiză se numeşte *analiză asimptotică.* In cadrul analizei asimptotice se consideră că un algoritm este mai eficient decât altul dacă ordinul de creştere al timpului de execuţie al primului este mai mic decât al celuilalt.

Există următoarele cazuri când ordinul de creştere a timpului de execuţie, nu e cel mai bun criteriu de apreciere a performantelor unui algoritm:

- dacă un program se rulează de puţine ori, se alege algoritmul cel mai uşor de implementat;
- dacă întreţinerea trebuie făcută de o altă persoană decât cea care l-a scris, un algoritm simplu, chiar mai puţin eficient, e de preferat unuia performant, dar foarte complex şi greu de înţeles;
- există algoritmi foarte eficienţi, dar care necesită un spaţiu de memorie foarte mare, astfel încât folosirea memoriei externe le diminuează foarte mult performanţele.

2.1. Notaţii asimptotice.

Fie $\mathbf{N}$ mulţimea numerelor naturale (pozitive său zero) şi $\mathbf{R}$ mulţimea numerelor reale. Notam prin $\mathbf{N}^+$ şi $\mathbf{R}^+$ mulţimea numerelor naturale, respectiv reale, strict pozitive, şi prin $\mathbf{R}^*$ mulţimea numerelor reale nenegative. Mulţimea $\{true, false\}$ de constante booleene o notăm cu $\mathbf{B}$. Fie $f : \mathbf{N} \to \mathbf{R}^*$ o funcţie arbitrară. Definim mulţimea

$$O(f) = \{t : \mathbf{N} \to \mathbf{R}^* \mid (\exists c \in \mathbf{R}^+)\ (\exists n_0 \in \mathbf{N})\ (\forall n \geq n_0)\ [t(n) \leq cf(n)]\}$$

Cu alte cuvinte, $O(f)$ (se citeşte "ordinul lui f") este mulţimea tuturor funcţiilor t mărginite superior de un multiplu real pozitiv al lui f, pentru valori suficient de mari ale argumentului. Vom conveni să spunem că t este în ordinul lui f (sau, echivalent, t este în $O(f)$, sau $t \in O(f)$) chiar şi atunci când valoarea $f(n)$ este negativă sau nedefinită pentru anumite valori $n < n_0$. În mod similar, vom vorbi despre ordinul lui f chiar şi atunci când valoarea $t(n)$ este negativă sau nedefinită pentru un număr finit de valori ale lui n; în acest caz, vom alege n_0 suficient de mare, astfel încât, pentru $n \geq n_0$, acest lucru să nu mai apară. De exemplu, vom vorbi despre ordinul lui $n/\log n$, chiar dacă pentru $n = 0$ şi $n = 1$ funcţia nu este definită. În loc de $t \in O(f)$, uneori este mai convenabil să folosim notaţia $t(n) \in O(f(n))$, subînţelegând aici că $t(n)$ şi $f(n)$ sunt funcţii.

Notaţia asimptotică defineşte o relaţie de ordine parţială între funcţii şi deci, între eficienţa relativă a diferiţilor algoritmi care rezolvă o anumită problemă. Vom da în continuare o interpretare algebrică a notaţiei asimptotice. Pentru oricare două funcţii $f, g : \mathbf{N} \to \mathbf{R}^*$, definim următoarea relaţie binară: $f \leq g$ dacă $O(f) \subseteq O(g)$. Relaţia "$\leq$" este o *relaţie de ordine parţiala* în mulţimea funcţiilor definite pe $\mathbf{N}$ şi cu valori în $\mathbf{R}^*$. Definim şi o *relaţie de echivalenta*: $f \equiv g$ dacă $O(f) = O(g)$.

În mulţimea $O(f)$ putem înlocui pe f cu orice altă funcţie echivalentă cu f. De exemplu, $\lg n \equiv \ln n \equiv \log n$ şi avem $O(\lg n) = O(\ln n) = O(\log n)$. Notând cu $O(1)$ ordinul funcţiilor mărginite superior de o constantă, obţinem ierarhia:

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(2^n)$$

Această ierarhie corespunde unei clasificări a algoritmilor după un criteriu al performanţei. Pentru o problemă dată, dorim mereu să obţinem un algoritm corespunzător unui ordin cât mai "la stânga". Astfel, este o mare realizare dacă în locul unui algoritm exponenţial găsim un algoritm polinomial.

Notaţia $O(f)$ este folosită pentru a limita superior timpul necesar unui algoritm, măsurând eficienţa algoritmului respectiv. Uneori este util să estimăm şi o limită inferioară a acestui timp. În acest scop, definim mulţimea

$$\Omega(f) = \{t : \mathbf{N} \to \mathbf{R}^* \mid (\exists c \in \mathbf{R}^+)\ (\exists n_0 \in \mathbf{N})\ (\forall n \geq n_0)\ [t(n) \geq cf(n)]\}$$

Există o anumită dualitate între notaţiile $O(f)$ şi $\Omega(f)$. Şi anume, pentru două funcţii oarecare $f, g : \mathbf{N} \to \mathbf{R}^*$, avem: $f \in O(g)$, dacă şi numai dacă $g \in \Omega(f)$.

O situaţie fericită este atunci când timpul de execuţie al unui algoritm este limitat, atât inferior cât şi superior, de câte un multiplu real pozitiv al aceleiaşi funcţii. Introducem notaţia

$$\Theta(f) = O(f) \cap \Omega(f)$$

numită *ordinul exact* al lui *f*. Pentru a compara ordinele a două funcţii, notaţia Θ nu este însă mai puternică decât notaţia O, în sensul că relaţia $O(f) = O(g)$ este echivalentă cu $\Theta(f) = \Theta(g)$.

Se poate întâmpla că timpul de execuţie al unui algoritm să depindă simultan de mai mulţi parametri. Această situaţie este tipică pentru anumiţi algoritmi care operează cu grafuri şi în care timpul depinde atât de numărul de vârfuri, cât şi de numărul de muchii. Notaţia asimptotică se generalizează în mod natural şi pentru funcţii cu mai multe variabile. Astfel, pentru o funcţie arbitrară $f : \mathbf{N} \times \mathbf{N} \to \mathbf{R}^*$ definim

$$O(f) = \{t : \mathbf{N} \times \mathbf{N} \to \mathbf{R}^* \mid (\exists c \in \mathbf{R}^+)\ (\exists m_0, n_0 \in \mathbf{N})\ (\forall m \geq m_0)\ (\forall n \geq n_0)\ [t(m, n) \leq cf(m, n)]\}$$

Similar, se obţin şi celelalte generalizări.

2.2. Operaţii asupra funcţiilor asimptotice

1. Dacă $T1(n)$ şi $T2(n)$ sânt timpii de execuţie a două secvenţe de program $P1$ şi $P2$, $T1(n)$ fiind $O(f(n))$, iar $T2(n)$ fiind $O(g(n))$, atunci timpul de execuţie $T1(n)+T2(n)$, al secvenţei $P1$ urmată de $P2$, va fi $O(\max(f(n),g(n)))$.
2. Dacă $T1(n)$ este $O(f(n))$ şi $T2(n)$ este $O(g(n))$, atunci $T1(n)*T2(n)$ este $O(f(n)*g(n))$.

3. Etapele analizei complexităţii

În analiza complexităţii unui algoritm se parcurg următoarele etape:

1. Se stabileşte dimensiunea problemei.
2. Se identifică operaţia de bază.
3. Se verifică dacă numărul de execuţii ale operaţiei de bază depinde doar de dimensiunea problemei. Dacă da, se determină acest număr. Dacă nu, se analizează cazul cel mai favorabil, cazul cel mai defavorabil şi (dacă este posibil) cazul mediu.
4. Se stabileşte clasa de complexitate căruia îi aparţine algoritmul.

Câteva reguli generale pentru evaluarea timpului de execuţie în funcţie de mărimea n a datelor de intrare, sânt:

1. Timpul de execuţie a unei instrucţiuni de asignare, citire sau scriere, este constant şi se notează cu $O(1)$.
2. Timpul de rulare a unei secvenţe de instrucţiuni e determinat de regula de însumare, fiind proporţional cu cel mai lung timp din cei ai instrucţiunilor secvenţei.
3. Timpul de execuţie a unei instrucţiuni *if - then - else* este suma dintre timpul de evaluare a condiţiei ($O(1)$) şi cel mai mare dintre timpii de execuţie ai instrucţiunilor pentru condiţia adevărată sau falsă.
4. Timpul de execuţie a unei instrucţiuni de ciclare este suma, pentru toate iteraţiile, dintre timpul de execuţie a corpului instrucţiunii şi cel de evaluare a condiţiei de terminare ($O(1)$).
5. Pentru evaluarea timpului de execuţie a unei proceduri recursive, se asociază fiecărei proceduri recursive un timp necunoscut $T(n)$, unde n măsoară argumentele procedurii; se poate obţine o relaţie recurentă pentru $T(n)$, adică o ecuaţie pentru $T(n)$, în termeni $T(k)$, pentru diferite valori ale lui k.
6. Timpul de execuţie poate fi analizat chiar pentru programele scrise în pseudocod; pentru secvenţele care cuprind operaţii asupra unor structuri de date, se pot alege câteva implementări şi

astfel se poate face comparaţie între performanţele implementărilor, în contextul aplicaţiei respective.

4. Analiza empirică a complexităţii algoritmilor

O alternativă la analiza matematică a complexităţii o reprezintă *analiza empirică.*

Aceasta poate fi utilă pentru: (i) a obţine informaţii preliminare privind clasa de complexitate a unui algoritm; (ii) pentru a compara eficienţa a doi (sau mai mulţi) algoritmi destinaţi rezolvării aceleiaşi probleme; (iii) pentru a compara eficienţa mai multor implementări ale aceluiaşi algoritm; (iv) pentru a obţine informaţii privind eficienţa implementării unui algoritm pe un anumit calculator.

Etapele analizei empirice. In analiza empirică a unui algoritm se parcurg de regulă următoarele etape:

1. Se stabileşte scopul analizei.
2. Se alege metrica de eficienţă ce va fi utilizată (număr de execuţii ale unei/unor operaţii sau timp de execuţie a întregului algoritm sau a unei porţiuni din algoritm.
3. Se stabilesc proprietăţile datelor de intrare în raport cu care se face analiza (dimensiunea datelor sau proprietăţi specifice).
4. Se implementează algoritmul într-un limbaj de programare.
5. Se generează mai multe seturi de date de intrare.
6. Se execută programul pentru fiecare set de date de intrare.
7. Se analizează datele obţinute.

Alegerea măsurii de eficienţă depinde de scopul analizei. Dacă, de exemplu, se urmăreşte obţinerea unor informaţii privind clasa de complexitate sau chiar verificarea acurateţei unei estimări teoretice atunci este adecvată utilizarea numărului de operaţii efectuate. Dacă însă scopul este evaluarea comportării implementării unui algoritm atunci este potrivit timpul de execuţie.

Pentru a efectua o analiză empirică nu este suficient un singur set de date de intrare ci mai multe, care să pună în evidenţă diferitele caracteristici ale algoritmului. În general este bine să se aleagă date de diferite dimensiuni astfel încât să fie acoperită plaja tuturor dimensiunilor care vor apare in practică. Pe de altă parte are importanţă şi analiza diferitelor valori sau configuraţii ale datelor de intrare. Dacă se analizează un algoritm care verifică dacă un număr este prim sau nu şi testarea se face doar pentru numere ce nu sunt prime sau doar pentru numere care sunt prime atunci nu se va obţine un rezultat relevant. Acelaşi lucru se poate întâmpla pentru un algoritm a cărui comportare depinde de gradul de sortare a unui tablou (dacă se aleg fie doar tabloul aproape sortate după criteriul dorit fie tablouri ordonate în sens invers analiza nu va fi relevantă).

În vederea analizei empirice la implementarea algoritmului într-un limbaj de programare vor trebui introduse secvenţe al căror scop este monitorizarea execuţiei. Dacă metrica de eficienţă este numărul de execuţii ale unei operaţii atunci se utilizează un contor care se incrementează după fiecare execuţie a operaţiei respective. Dacă metrica este timpul de execuţie atunci trebuie înregistrat momentul intrării în secvenţa analizată şi momentul ieşirii. Majoritatea limbajelor de programare oferă funcţii de măsurare a timpului scurs între două momente. Este important, în special în cazul în care pe calculator sunt active mai multe taskuri, să se contorizeze doar timpul afectat execuţiei programului analizat. În special dacă este vorba de măsurarea timpului este indicat să se ruleze programul de test de mai multe ori şi să se calculeze valoarea medie a timpilor.

După execuţia programului pentru datele de test se înregistrează rezultatele iar în scopul analizei fie se calculează mărimi sintetice (media, abaterea standard etc.) fie se reprezintă grafic perechi de puncte de forma (dimensiune problema, măsură de eficienţă).

5. Exemple

1. Mai jos se prezintă modul de evaluare a timpului de execuţie a procedurii de sortare a elementelor unui tablou de dimensiune *n* prin metoda "bubble":

```
type int_array = array[1..n] of integer;
procedure bubble (var a: int_array); {sortare crescătoare}
  var i, j, temp: integer;
  begin
(1) for i:=1 to n - 1 do
(2)   for j:=n downto i + 1 do
```

(3) **if** *a*[*j* - 1] > *a*[*j*] **then begin**
(4) *temp*:=*a*[*j* - 1];
(5) *a*[*j* - 1]:=*a*[*j*];
(6) *a*[*j*]:=*temp*
end
end; {bubble}

Timpii de rulare pentru instrucţiunile de asignare (4),(5) si (6) sânt $O(1)$, deci pentru secvenţa (4)-(6), timpul este $O(\max(1,1,1)) = O(1)$.

Pentru instrucţiunea **if - then** (3), timpul este suma dintre cel pentru evaluarea condiţiei, $O(1)$ şi cel al secvenţei ce se execută la condiţie adevărată, tot $O(1)$, calculat mai sus, deci $O(1)$, acesta fiind deci şi timpul pentru o iteraţie a instrucţiunii **for** (2).

Pentru cele $n - i$ iteraţii ale lui **for** (2), timpul de execuţie este $O((n-i)*1) = O(n - i)$.

Pentru instrucţiunea **for** (1), având n-1 iteraţii, timpul de execuţie este:

$$S(i=1, n-1)\,(n-i) = (n-1) + (n-2) + \ldots + (n-n+1) = n^2/2 - n/2,$$

deci este $O(n^2)$.

2. Modalitatea de evaluare a timpului de execuţie al unei proceduri recursive, este ilustrată prin cea a funcţiei de calcul al factorialului:

Function *factorial*(*n*: integer): integer;
begin
(1) **if** $n \le 1$ **then**
(2) **factorial**:= 1
else
(3) **factorial**:= *n***factorial*(*n* - 1)
end; {factorial}

Dimensiunea intrării este aici n, valoarea numărului al cărui factorial se calculează şi se notează cu $T(n)$, timpul de rulare al funcţiei *factorial*(n).

Timpul de rulare pentru liniile (1) si (2) este $O(1)$, iar pentru linia (3) este $O(1) + T(n - 1)$, deci cu constantele c si d neprecizate:

$T(n) = c + T(n - 1)$, pentru $n > 1$ sau $T(n) = d$, pentru $n \le 1$.

Pentru $n > 2$, expandând pe $T(n - 1)$, se obţine $T(n) = 2c + T(n - 2)$, pentru $n > 2$.

Pentru $n > 3$, expandând pe $T(n - 2)$, se obţine $T(n) = 3c + T(n - 3)$, pentru $n > 3$.

Deci, în general $T(n) = ic + T(n - i)$, pentru $n > i$.

În final, când $i = n - 1$, se obţine $T(n) = (n - 1)c + T(1) = (n - 1)c + d$.

Deci $T(n)$ este $O(n)$.

6. Numerele lui Fibonacci

Şirul lui Fibonacci este definit prin următoarea recurenţa:

$$\begin{cases} f_0 = 0;\ f_1 = 1 \\ f_n = f_{n-1} + f_{n-2} \quad \text{pentru} \quad n \ge 2 \end{cases}$$

Acest celebru şir a fost descoperit în 1202 de către Leonardo Pisano (Leonardo din Pisa), cunoscut sub numele de Leonardo Fibonacci. Cel de-al n-lea termen al şirului se poate obtine direct din definiţie:

1. function *fib*1(*n*)
if *n* < 2 **then return** *n*
else return *fib*1(*n*-1) + *fib*1(*n*-2)

Această metodă este foarte ineficienta, deoarece recalculează de mai multe ori aceleaşi valori. Urmează o altă metodă, mai performantă, care rezolvă aceeaşi problemă.

2. function *fib*2(*n*)
    $i \leftarrow 1; j \leftarrow 0$
    for $k \leftarrow 1$ **to** *n* **do** $j \leftarrow i + j$
                $i \leftarrow j - i$
    return *j*

Mai există un algoritm :

3. function *fib*3(*n*)
    $i \leftarrow 1; j \leftarrow 0; k \leftarrow 0; h \leftarrow 1$
    while $n > 0$ **do**
        if *n* este impar **then** $t \leftarrow jh$
                $j \leftarrow ih+jk+t$
                $i \leftarrow ik+t$
        $t \leftarrow h^2$
        $h \leftarrow 2kh+t$
        $k \leftarrow k^2+t$
        $n \leftarrow n$ **div** 2
    return *j*

SARCINA DE BAZĂ:

1. Efectuaţi analiza empirică a algoritmilor propuşi în paragraful 6.
2. Determinaţi relaţia ce determină complexitatea temporală pentru aceşti algoritmi.
3. Determinaţi complexitatea asimptotică a algoritmilor.
4. Faceţi o concluzie asupra lucrării efectuate.

Întrebări de control:

1. Enumeraţi factorii ce influenţează timpul de execuţie al algoritmului.
2. Când timpul de execuţie al algoritmului este dat de o relaţie de recurenţă ?
3. Care sunt regulile generale pentru evaluarea timpului de execuţie?
4. Care sunt etapele analizei empirice şi în care cazuri se face analiza empirică a algoritmilor?
5. Ce proprietăţi posedă funcţiile asimptotice?